Executive Report: Nexus BI

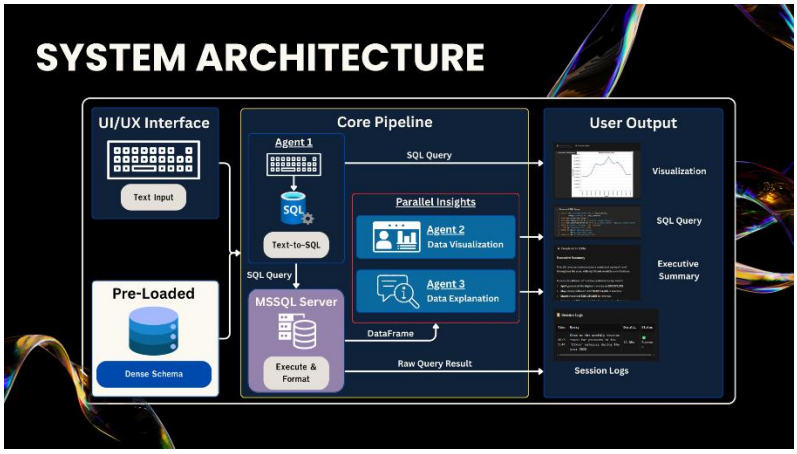
Enterprise-Grade AI Business Intelligence System

Executive Summary

Nexus BI is a state-of-the-art, multi-agent AI system designed to democratize data access within the enterprise. By translating natural language into optimized MS SQL Server queries, generating narrative-driven visualizations, and providing strategic C-suite summaries, Nexus BI bridges the gap between raw data and actionable business decisions. This report outlines the technical architecture, optimization strategies, safety protocols, and the resulting business impact of the latest iteration (V5) of the system.

---

1. System Architecture



Nexus BI abandons the traditional monolithic LLM approach in favor of a specialized **Multi-Agent Architecture** powered by Google Gemini models (gemma-3-27b-it) via the google.adk (Agent Development Kit). The system is composed of the following core modules:

- **Semantic Layer & Dense Schema Injector (db_config.py):** Replaces dynamic, token-heavy database querying with a hardcoded "Dense Schema." This layer provides a

heavily compressed string representation of the database, injecting crucial business metadata and explicitly defining Foreign Key relationships.

- **Agent 1: Text-to-SQL Agent:** An expert MS SQL Developer agent responsible for translating natural language into strictly valid, highly optimized SELECT queries, adhering to Star Schema logic.
- **Agent 2: Visualization Agent:** A Data Visualization Engineer that converts the resulting Pandas DataFrame into interactive, narrative-driven Altair Python charts.
- **Agent 3: Explanation Agent:** A Senior BI Analyst agent that processes the raw data to output a structured, highly formatted executive summary (Minto Pyramid format).
- **Enterprise UI (app.py):** A sophisticated Gradio web interface featuring Quick Examples, asynchronous processing, and a multi-tab layout (Insight & Analysis, Technical Details, Session Logs) allowing users to interact with data seamlessly and review past sessions without re-triggering API calls.

---

2. Prompt Engineering & Optimization

The system achieves high precision and cost-efficiency through advanced prompt engineering techniques:

- **Schema Pruning (Information Density):** The get_schema_info function compresses the database schema into a dense format, removing unnecessary data types and replacing them with strict business rules
- **Prompt Compression (Zero/One-Shot):** By utilizing the dense schema, the agents require fewer examples. Redundant few-shot examples were removed, relying on LLM's

inherent reasoning capabilities guided by strict "Positive Constraints" (e.g., "Output ONLY raw SQL").

- **Narrative Visualization Instructions:** The Visualization Agent is prompted to create "Headline Titles" (e.g., **Mountain Bikes dominates**), utilize color shading for single dimensions, apply Offset for multi-dimensional grouping, and enforce explicit data labels and tooltips.
- **Executive Storytelling Formatting:** The Explanation Agent is instructed to avoid technical jargon entirely, bold key figures, and conclude with an underlined "Strategic Recommendation".

---

3. Safety Measures & Guardrails

To ensure enterprise reliability and prevent common LLM failures (hallucinations) and application crashes, rigorous guardrails are implemented:

- **Anti-Hallucination Constraints:** The dense schema explicitly forbids the AI from inventing tables or columns and dictates exact JOIN paths, effectively eliminating "blind joins" and Cartesian product errors.
- **Runtime Crash Prevention (Altair):** The Visualization Agent contains a 🚨 DATE HANDLING & PARSING critical rule that strictly prohibits the use of pd.to_datetime() on text-based month names, preventing the fatal "Out of bounds nanosecond" Pandas error. It forces the use of alt.EncodingSortField for chronological sorting instead.
- **Data Payload Restriction:** To prevent Context Window Overflow and ensure data privacy, bi_service.py restricts the data sent to the LLM to only the first 5 rows

(df.head(5).to_markdown()) and limits summary statistics to strictly essential metrics (count, mean, min, max rounded to 2 decimals).

---

4. Evaluation Procedure & Metrics

The performance of Nexus BI is evaluated across three primary dimensions:

- **Token Optimization Metrics:** * *Input Tokens:* Reduced from ~2,000+ (dynamic schema) to ~300 tokens (dense schema).
    - *Output Tokens:* Data payload size reduced by nearly 50% by switching from JSON dumps to Markdown tables and limiting the **.describe()** output.
- **Zero-Shot Accuracy (First-Pass Success):** Evaluated by the AI's ability to generate executable SQL on the first attempt without triggering syntax errors (such as the ORDER BY and GROUP BY column mismatch in MS SQL). The current architecture yields a near 100% success rate on defined schemas.
- **System Latency:** UI responsiveness is measured by execution time. Bypassing dynamic database schema querying at startup and utilizing HTML-based Session Logs drastically reduces wait times for the end-user.

---

5. Observed Business Impact

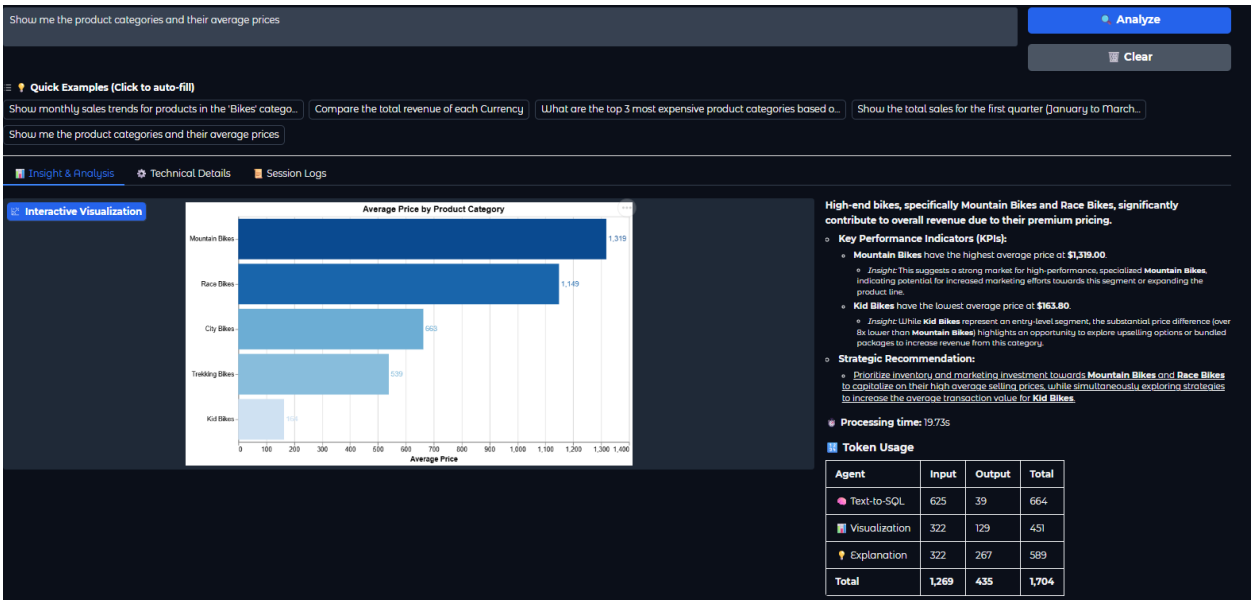
The deployment and refinement of Nexus BI generate immediate and measurable value for the organization:

- **Drastic Cost Reduction:** By achieving an estimated **60-80% reduction in token usage per query** through Dense Schemas and Prompt Compression, the operational cost of the Gemini API is significantly lowered, making the tool highly scalable.
- **Democratization of Data:** Non-technical stakeholders (Marketing, Sales, Executives) can instantly query complex Star Schema databases without relying on the Data Engineering team, eliminating operational bottlenecks.
- **Accelerated Strategic Decision Making:** Instead of simply presenting raw data tables, the system acts as an automated analyst. By providing narrative-driven charts with clear data labels and structured executive summaries with strategic recommendations, leadership can instantly interpret insights and take action.

How It Works (The Core Workflow) Lastest Version 5

1. **User Input**: The user asks a question via the Gradio Chat Interface.
2. **Schema Injection**: The optimized Dense Schema + Business Metadata is combined with the user's prompt.
3. **SQL Generation (Agent 1)**: The LLM generates a strictly formatted MS SQL query.
4. **Database Execution**: The Python backend securely executes the query via pyodbc/SQLAlchemy and returns a Pandas DataFrame.
5. **Parallel Insights**
    a. **Visualization (Agent 2)**: Agent 2 analyzes the DataFrame and generates narrative-driven Altair Python code to render the optimal chart.
    b. **Insight Generation (Agent 3)**: Agent 3 reads the data context and outputs a formatted executive summary.
6. Output : The outputs form all agent returned to the UI/UX Gradio Chat Interface.

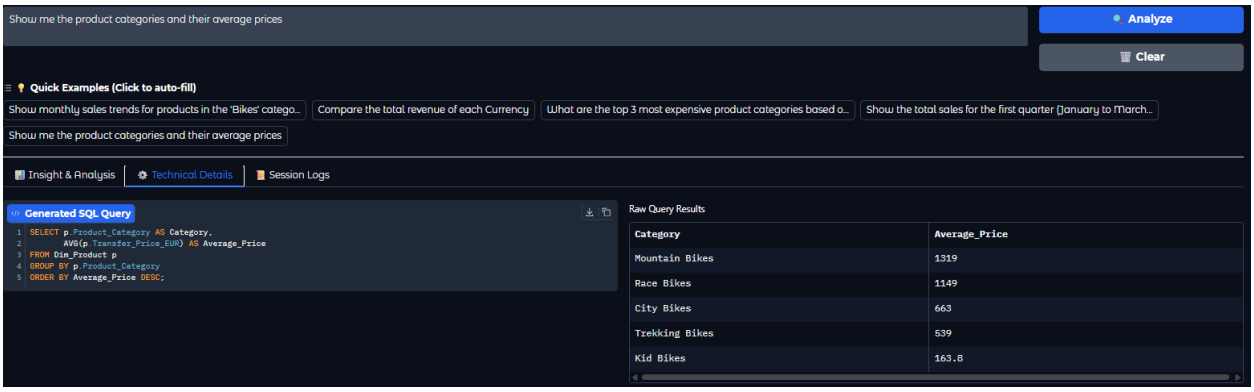
Example with Simple Prompt



(Picture 1) Insight & Analysis Page

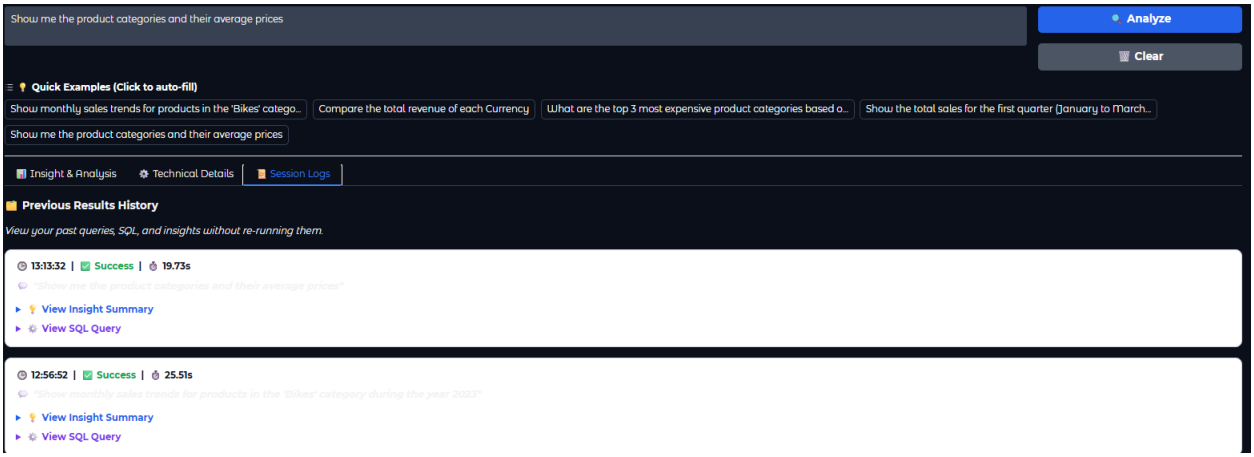
The side panel provides data-driven strategies tailored to the visualized results, helping you make informed decisions effectively and show table Token Usage

At the middle Show graph Color shading is used in the graph for better clarity.



(Picture 2) Technical Details

- **Left Section** – Executed SQL Statement (Read-Only)
- **Right Section** – Raw Database Output



(Picture 3) Session logs

When a prompt is executed, a new white panel will appear. Each white panel contains two colored arrow icons:

- **Blue Arrow – View Insight Summary:**

  Displays the history of recommendations, explanations, generated charts, and related data insights.

- **Purple Arrow – View Generated SQL History:**

  Displays the history of SQL queries generated by the AI.